Tema 2 · Criptografía

SPD · Curso 2026/27 · Prof. Teodoro Rojo
Apuntes a partir de *SPD - 2. Criptografía*. Muchas diapositivas de este tema son esquemas (imágenes); aquí están explicados con texto y con **ejemplos numéricos comprobados**. "Nota" = aclaración o ejemplo añadido.

Índice

1. Introducción: servicios, ataques y mecanismos
2. Modelo y tipos de cifradores
3. Criptoanálisis y seguridad del cifrado
4. Bases teóricas
5. Otros sistemas: AE, AEAD, FPE, TE
6. Ubicación de los mecanismos en la pila de protocolos
7. Criptografía clásica
8. Cifrado simétrico: DES, 3DES, AES, IDEA
9. Modos de operación: ECB, CBC, CFB…
10. Distribución de claves: KDC
11. Cifrado de clave pública: RSA, DSA, ECC…
12. Funciones hash y MAC
13. Firma digital
14. Diffie-Hellman
15. Certificados y jerarquía de certificación
16. Chuleta final
17. Preguntas y ejercicios

---

1. Introducción

1.1 Servicios de seguridad (modelo OSI)

- **Confidencialidad**
- **Integridad**
- **Autentificación**
- **Imposibilidad de rechazo** (no repudio): con **prueba de origen** / con **prueba de entrega**
- **Control de acceso**

1.2 Ataques

Tipo	Qué hace	Ejemplos	Servicio atacado
Pasivo	El intruso **escucha** sin modificar	Interceptar el contenido; **análisis de tráfico** (quién habla con quién, cuánto, cuándo)	Confidencialidad
Activo	El intruso **altera, crea o falsea** datos	Modificar mensajes, **suplantar** a un interlocutor, **interrumpir** el diálogo (**DoS**)	Integridad, autenticidad, disponibilidad

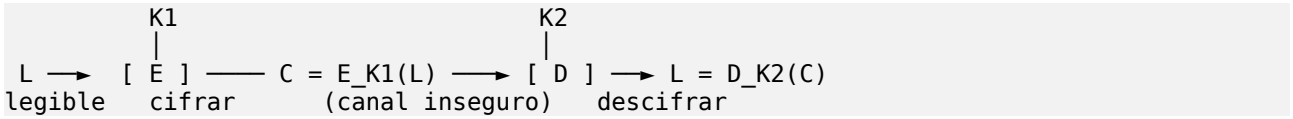
Nota: Los ataques pasivos son difíciles de **detectar** → se **previenen** (cifrado). Los activos son difíciles de **prevenir** → se **detectan** (integridad, autenticación).

1.3 Mecanismos de seguridad (importante)

Mecanismo	Qué es	Servicio que soporta	Se basa en
Cifrado	Alteración de la información que la hace **irreconocible** al intruso y **reversible** en destino	Confidencialidad (y es **base de todos los demás**)	Algoritmo + clave
Integridad de datos	Secuencia cifrada obtenida de los datos (**huella digital o MAC**). Para mensajes de varios bloques: **numeración secuencial, encadenamiento criptográfico, sellos de tiempo**	Integridad	**Funciones hash**
Firma digital	Secuencia que se adjunta al mensaje y permite verificar **identidad del originante e integridad**	Autentificación con prueba de origen + integridad	**Hash + clave privada**
Pruebas de identidad	Intercambio entre interlocutores (**reto-respuesta**) para verificar que cada uno es quien dice	Autentificación y control de acceso	—
Entidades fedatarias	Terceros de **confianza** que actúan como **notarios**; a veces también **CA** o **KDC**	**No repudio**	—

2. Modelo y tipos de cifradores

2.1 Modelo de sistema de cifrado



- **L** = texto legible (*plaintext*), **C** = texto cifrado, **E/D** = algoritmos de cifrado/descifrado, **K1/K2** = claves.
- **Principio de Kerckhoffs:** en sistemas comerciales los algoritmos E y D son **públicos** → **la seguridad reside en las claves**.

2.2 Simétricos vs asimétricos (importante)

	Simétrico (clave privada / única)	Asimétrico (clave pública)
Claves	**K1 = K2 = K**	**K1 ≠ K2** (par pública/privada)
N° de claves para *m* usuarios	**m(m−1)/2**	**2m**
Problema principal	**Distribución de la clave**	Lentitud, claves largas
Autenticación	Difícil	**Fácil** de implementar
Velocidad	**Muy rápido**	Lento
Ejemplos	DES, 3DES, AES, IDEA,	RSA, DSA, ECDSA, ElGamal

	Simétrico (clave privada / única)	Asimétrico (clave pública)
	ChaCha20	

Nota: Ejemplo: 26 alumnos que quieren hablar todos con todos → simétrico: 26·25/2 = **325 claves**; asimétrico: 2·26 = **52 claves**.

2.3 Requisitos de un buen cifrador

- E y D **computacionalmente eficientes** para todas las claves.
- **No inferencia de la clave** a partir del cifrado, ni a partir de pares legible-cifrado (**principio de máxima entropía**).

2.4 Cifradores de bloque vs de flujo

	Bloque	Flujo
Unidad	Bloques de **longitud fija** (64 o 128 bits)	Bit a bit u octeto a octeto
Clave	**La misma** en todas las transformaciones	**Distinta** en cada transformación (secuencia cifrante)
Nota	Los bloques se encadenan (**encadenamiento criptográfico**) para garantizar integridad y eliminar redundancia	Ideal para flujos continuos
Ejemplos	DES, AES, IDEA	RC4, ChaCha20, Vernam

3. Criptoanálisis y seguridad del cifrado

Criptoanálisis: obtener el legible L a partir del cifrado C **sin conocer la clave** (a veces tampoco el algoritmo).

3.1 Tipos de ataque (de menos a más información para el atacante) (importante)

Ataque	Siglas	El atacante dispone de...
Solo texto cifrado	**COA** (*Ciphertext-only*)	Solo fragmentos o todo el **cifrado**
Texto conocido	**KPA** (*Known-plaintext*)	**Parejas legible-cifrado**
Texto legible elegido	**CPA** (*Chosen-plaintext*)	Puede obtener el cifrado de **cualquier legible que elija**
Texto cifrado elegido	**CCA** (*Chosen-ciphertext*)	Puede **cifrar y descifrar** lo que quiera

Herramientas clave: **análisis estadístico** (frecuencias de letras, bigramas, trigramas, palabras probables) y **potencia de cálculo**.

3.2 Seguridad incondicional vs computacional (importante)

	Incondicional	Computacional
Idea	El criptoanalista **no tiene información suficiente** para romperlo, por mucho cómputo que tenga	Podría romperlo, pero el **nº de pruebas es tan grande** que es inviable hoy y en un futuro próximo
Cómo	Clave **aleatoria** de **al menos la misma longitud que el mensaje** y **no reutilizada**	Claves suficientemente largas
Ejemplo	**Cifrado de Vernam (one-time pad)**	AES, RSA...
Ataques	—	**Fuerza bruta**, **diccionario**, o explotar **debilidades** (reducen

	Incondicional	Computacional
		el n° de pruebas)

Nota: En la diapositiva pone "claves aleatorias de al menos la misma longitud que **la clave**": es una errata, debe ser "**que el mensaje** (legible)".

3.3 Longitudes de clave recomendadas (diapositiva)

Sistema	Longitud mínima
Clave simétrica	**≥ 256 bits**
Clave pública RSA	**≥ 4096 bits**
Clave pública ECC	**≥ 512 bits**

Nota: Son recomendaciones conservadoras del profesor (para el examen, quédate con estas). Como referencia, NIST considera hoy seguros AES-128, RSA-3072 y ECC-256 (equivalentes a ~128 bits de seguridad).

---

4. Bases teóricas

1. **Teoría de la información — Shannon (1948).** Herramientas para analizar sistemas de **clave privada**. Estudia la **entropía H(x)**, la **redundancia del lenguaje** y las claves. Propone **distribuir y encubrir la redundancia** del lenguaje mediante **uso repetido de sustituciones y transposiciones** (confusión y difusión) → base de **DES, AES**.
2. **Teoría de números / complejidad — Diffie y Hellman (1976).** Basada en **funciones unidireccionales**:
   - **Función unidireccional:** f es fácil (P) pero f^{-1} es difícil (NP). Ej.: `2^x mod 13 = 3` → x = 4 (calcular 2^4 mod 13 es trivial; despejar x en general es el **logaritmo discreto**, difícil).
   - **Función unidireccional con trampa:** f^{-1} es difícil **salvo que conozcas una información extra (la clave)**. → base de **RSA, ECDSA**.

Nota: Confusión = relación compleja entre clave y cifrado (sustituciones). **Difusión** = cada bit del legible afecta a muchos bits del cifrado (transposiciones/permutaciones).

---

5. Otros sistemas de cifrado

5.1 AE — *Authenticated Encryption*

Cifrado simétrico **reforzado con integridad**: devuelve el cifrado **C** y una **etiqueta de autentificación T** (*AuthTag*): `(C, T) = AE_K(L)`. Al descifrar, **si la etiqueta no coincide, el mensaje no se procesa** y se devuelve error. Garantiza **confidencialidad + integridad + autenticidad** del interlocutor.

Variantes:

Variante	Cómo funciona	Nota Nota:
EtM (*Encrypt-then-MAC*)	Se cifra L con Key1 y se calcula el MAC **sobre el cifrado** con Key2 → C ‖ MAC	La **más segura**; usada en IPsec
E&M (*Encrypt-and-MAC*)	Se cifra L y, en paralelo, se calcula el MAC **sobre el legible** → C ‖ MAC	Usada en SSH
MtE (*MAC-then-Encrypt*)	Se calcula el MAC del legible, se concatena L ‖ MAC y **se cifra todo**	Usada en TLS antiguo (≤1.2); vulnerable a *padding oracle*

5.2 AEAD — *AE with Associated Data*

Como AE pero además usa **datos asociados** que conocen ambos extremos y **no se cifran** pero sí se autentican (p. ej. identificador de sesión, cabeceras). A veces también un **nonce** (valor de un solo uso) para que no puedan descifrarse intercambios antiguos si se revela la clave.

```
key, nonce, plaintext, associated data ⟶ [AES-GCM encrypt] ⟶ ciphertext + tag
key, nonce, ciphertext, associated data ⟶ [AES-GCM decrypt] ⟶ plaintext  ó  error
```

Ejemplos: **AES-GCM**, **AES-GCM-SIV**, ChaCha20-Poly1305.

5.3 FPE — *Format-Preserving Encryption*

El cifrado **mantiene el formato** del original (n° de tarjeta, IP, PDF...). Ej.: `10.0.0.5 → 185.47.239.6`. **Nota:** Útil para cifrar datos en bases de datos sin cambiar el esquema.

5.4 TE — *Tweakable Encryption*

Cifrado simétrico "retocado": además de la clave **K** usa un **tweak T** específico (de cada usuario/sector) que impide "clonar" el entorno: `C = TE_K,T(L)`. Ej.: **cifrado de discos** (el tweak es el n° de sector, p. ej. AES-XTS).

---

6. Ubicación de los mecanismos

Nivel	Dónde se sitúa la seguridad	Ejemplos
a) **Red**	Dentro de IP	**IPsec (AH, ESP)**
b) **Sesión/transporte**	Entre TCP y la aplicación	**SSL/TLS**
c) **Aplicación (I)**	Versiones seguras del protocolo	**S-HTTP, S/MIME**
c) **Aplicación (II)**	Por encima del protocolo de aplicación	**SET** (pagos), **PGP** (correo)

Nota: Cuanto más **abajo**, más **transparente** para las aplicaciones (IPsec protege todo el tráfico); cuanto más **arriba**, más **específico** y con más control (PGP firma un correo concreto).

---

7. Criptografía clásica

Sistemas tradicionales de cifrado de texto, basados en **sustitución** (cambiar letras por otras) y **transposición** (cambiar el orden de las letras).

7.1 Sustitución monoalfabética

- Una **permutación del alfabeto**: la clave es la secuencia sustituta.
- **Vulnerable al análisis de frecuencias** (en español la E, A, O son las más frecuentes). Cifrar dos veces **no aporta nada** (sigue siendo otra monoalfabética).
- **Cifrado César:** caso particular, **desplazamiento fijo** (la clave). Según Suetonio, César usaba **desplazamiento 3**.

Nota: Ejemplo César (k = 3): `HOLA → KROD`. Solo hay 25 claves posibles → fuerza bruta inmediata.

7.2 Sustitución polialfabética · Vigenère

- **Varias** sustituciones monoalfabéticas usadas **de forma periódica**: con S alfabetos, los caracteres i, i+S, i+2S... se cifran igual.
- **Vigenère:** la clave es una palabra; cada letra indica el desplazamiento a aplicar. (Creado en el s. XVI por **Giovan Battista Bellaso**; usado hasta la I Guerra Mundial).
- **Dificulta** (no impide) el análisis de frecuencias.

Ejemplo de la diapositiva — clave **DUH** (desplazamientos 3, 20, 7), legible "THEY DRINK THE TEA":

```
Legible:  T  H  E  Y  D  R  I  N  K  T  H  E  T  E  A
Clave:    D  U  H  D  U  H  D  U  H  D  U  H  D  U  H
Despl.:   3 20  7  3 20  7  3 20  7  3 20  7  3 20  7
Cifrado:  W  B  L  B  X  Y  L  H  R  W  B  L  W  Y  H
```

→ **`WBLB XYLHR WBL WYH`** ✓ (comprobado). Fórmula: `C_i = (L_i + K_(i mod S)) mod 26`.

Nota: Fíjate: "THE" aparece dos veces y se cifra igual (WBL) porque coincide en la misma posición del ciclo de clave → esa repetición es la que explota el **método Kasiski** para averiguar la longitud de la clave.

7.3 Suma de texto y Vernam

- **Suma de texto:** polialfabética con desplazamiento usando **un texto como clave** (normalmente periódico).
- Si la clave es **aleatoria y tan larga como el legible** → **incondicionalmente segura** → **cifrado de Vernam**.
- **Vernam (one-time pad):** cifrado **de flujo**; clave **binaria aleatoria, no reutilizable**, de la misma longitud que el legible. Se combina con **XOR (or-exclusiva)**: `C = L ⊕ K` y `L = C ⊕ K`.

Nota: Ejemplo: `L = 1011`, `K = 0110` → `C = 1101`; y `1101 ⊕ 0110 = 1011` ✓. Si se **reutiliza** la clave: `C1 ⊕ C2 = L1 ⊕ L2` → se filtra información.

7.4 LFSR (*Linear Feedback Shift Register*)

- Alternativa a las claves verdaderamente aleatorias de Vernam (que son difíciles de generar y distribuir).
- Registro de desplazamiento **realimentado** que genera secuencias **pseudoaleatorias** de gran periodo.
- El receptor configura el **mismo LFSR** (mismos coeficientes de realimentación **Ci** y estado inicial) para regenerar la secuencia y descifrar.

7.5 Transposición

- **Transposición simple:** se **reordenan las posiciones** de las letras. Se detecta por estadística (las frecuencias de letras **no cambian**).
- **Cifrado Nihilist:** combinación de transposición **horizontal y vertical** (se escribe en una matriz y se permutan filas y columnas según una clave).

Nota: Ejemplo de transposición por columnas: escribimos "ATACAR AL AMANECER" en filas de 4 y leemos por columnas:

```
A T A C
A R A L
A M A N
E C E R    → leído por columnas: AAAE TRMC AAAE CLNR
```

---

8. Cifrado simétrico

- **Misma clave** para cifrar y descifrar. Algoritmos **públicos** → la seguridad está en la **longitud de la clave** (64, 128, 192, 256 bits). **Muy eficientes.**
- Algoritmos: **DES, 3DES, AES, IDEA**, Blowfish, Twofish, Camellia, CAST, Skipjack, **RC4 y ChaCha (flujo)**…
- **Inconvenientes:**
  - **Distribución segura** de la clave.
  - **Crecimiento del nº de claves**: N(N−1)/2.
  - **Desconfianza** entre interlocutores (ambos tienen la misma clave → no hay no-repudio).

8.1 DES (*Data Encryption Standard*) (importante)

Característica	Valor
Origen	**IBM** (sucesor de *Lucifer*). Estándar **NBS en 1977**; ISO lo recoge como **DEA1**
Bloque	**64 bits**

Característica	Valor
Clave	**56 bits efectivos + 8 de paridad** (64)
Estructura	**16 iteraciones** (rondas) con sustituciones (**cajas S**), transposiciones y XOR

Nota: Esquema general (red de Feistel): permutación inicial (IP) → el bloque se divide en mitades $\mathbf{L_0}$ **y** $\mathbf{R_0}$ de 32 bits → 16 rondas: $L_i = R_{i-1}$ y $R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$, donde *f* expande R a 48 bits, hace XOR con la subclave K_i (48 bits), pasa por las **8 cajas S** (48→32 bits) y una permutación P → permutación final (IP^{-1}). Las 16 subclaves K_i se derivan de la clave de 56 bits por rotaciones y selecciones. Descifrar = mismo proceso con las subclaves en **orden inverso**.

Efecto sobre los datos (efecto avalancha): no hay correlación entre L y C ni entre K y C: al cambiar **un bit** de L o de K, **cada bit de C cambia con probabilidad 1/2**.

Objeciones al DES:

- **Clave corta (56 bits)** → 2^{56} = **72.057.594.037.927.936** claves. Una máquina con 10^6 decodificadores en paralelo (1 µs cada uno) las prueba en **~24 h**. Máquinas reales: **COPACOBANA**, **RIVYERA (2008, 128 FPGAs Spartan-3)**.
- **Falta de explicación** de algunos principios de diseño (**cajas S**, generación de subclaves).
- **Claves inapropiadas: 4 débiles** (todas las subclaves K_i iguales) y **12 semidébiles** (solo 2 subclaves distintas).

8.2 DES múltiple · 3DES (importante)

Esquema	Claves	Longitud equivalente
2DES: E_K1 → E_K2	2	**57 bits** (no estandarizado)
3DES (EDE, 2 claves): E_K1 → **D**_K2 → E_K1	2	**112 bits**
3DES (EDE, 3 claves): E_K1 → D_K2 → E_K3	3	**168 bits**

Nota: ¿Por qué 2DES solo da 57 bits y no 112? Por el ataque **meet-in-the-middle**: el atacante cifra L con todas las K1 y descifra C con todas las K2, y busca coincidencias en el medio → $2 \cdot 2^{56} = 2^{57}$ operaciones. **Nota: ¿Por qué E-D-E y no E-E-E?** Por **compatibilidad**: si K1 = K2, E_K1(D_K1(E_K1(L))) = E_K1(L) → equivale a DES simple.

8.3 AES (*Advanced Encryption Standard*) (importante)

Característica	Valor
Origen	Algoritmo **Rijndael** de los belgas **Rijmen y Daemen**; ganador del concurso del **NIST**; estándar **FIPS PUB 197 (2001)**
Bloque	**128 bits**
Clave	**128, 192 o 256 bits**
Estado	Matriz **4×4 octetos**
Rondas	**10 / 12 / 14** según clave 128 / 192 / 256
Operaciones por ronda	**SubBytes** (sustitución con S-box), **ShiftRows** (desplazamiento de filas), **MixColumns** (mezcla de columnas), **AddRoundKey** (XOR con la subclave)

Nota: La última ronda no tiene MixColumns.

AES-NI (*New Instructions*): instrucciones específicas en procesadores x86-64 de **Intel y AMD** que ejecutan AES en **hardware dedicado** → hasta **un orden de magnitud** más rápido y sin sobrecargar la CPU. A veces hay que **activarlo en la BIOS**.

8.4 IDEA (*International Data Encryption Algorithm*)

- **1992**, Xuejia **Lai** y James **Massey** (ETH Zúrich).

- Bloque **64 bits**, clave **128 bits**.
- **8 iteraciones**: XOR, sumas módulo 2^{16} (64K) y multiplicaciones módulo $2^{16}+1$, guiadas por **6 subclaves** por iteración.
- Usado en **PGP**.

8.5 Comparativa rápida

	DES	3DES	AES	IDEA
Bloque	64	64	**128**	64
Clave	56	112 / 168	128 / 192 / 256	128
Rondas	16	48 (3×16)	10 / 12 / 14	8
Estado	Roto	Obsoleto	**Estándar actual**	Legado (PGP)

9. Modos de operación

Los cifradores de bloque trabajan con bloques fijos; para un mensaje largo hay que **trocearlo** y cifrar cada bloque. El **encadenamiento criptográfico** sirve para:

- **Detectar ataques a la integridad** (eliminar, insertar, reordenar o reutilizar bloques).
- **Encubrir redundancias** del legible.

Modos: **ECB, CBC, CFB, OFB, CTR, GCM...**

Modo	Fórmula	Ventajas	Problemas
ECB (*Electronic Codebook*) — uso directo	$C_i = E_K(L_i)$	Simple, paralelizable	**Legible repetido → cifrado repetido**; se pueden alterar/reordenar bloques. Ej. **imagen del pingüino Tux**: cifrada en ECB **se sigue viendo la silueta** (solo cambian los colores)
CBC (*Cipher Block Chaining*)	$C_i = E_K(L_i \oplus C_{i-1}), C_0 = IV$	Oculta patrones	**Inicialización** (necesita **IV**); **propagación de errores** al bloque siguiente; no paralelizable al cifrar
CFB (*Cipher Feedback*) — funciona como **cifrado de flujo**	$C_i = L_i \oplus E_K(C_{i-1})$ (por segmentos de 8 bits)	No necesita *padding*; flujo	**Inicialización**; propagación de errores (**9 octetos** en CFB-8 con DES)
OFB (*Output Feedback*)	$O_i = E_K(O_{i-1}), C_i = L_i \oplus O_i$	Errores no se propagan	El IV no debe repetirse nunca
GCM (*Galois/Counter Mode*)	Modo contador + autenticación (tag)	**AEAD**, paralelizable, rápido	Nonce nunca repetido

Nota: Pregunta típica: *¿Por qué no se debe usar ECB?* → Porque no oculta patrones: bloques iguales dan cifrados iguales.

---

10. Distribución de claves · KDC

Problema en sistemas simétricos: cada pareja comparte una clave; hay que **mantenerlas** y **distribuirlas** por una red insegura (un intruso podría interceptarlas). Además: claves robustas y distintas, protegerlas, autenticar al interlocutor...

Solución: un **KDC (*Key Distribution Center*)**, entidad de confianza que **almacena, gestiona y distribuye** claves.

Escenario KDC (importante)

Cada usuario comparte con el KDC una **clave maestra única** (Kmx para X, Kmy para Y). Para hablar, obtienen una **clave de sesión Ks**:

Paso	Mensaje	Significado
1	X → KDC: `[X, Y, T₁]`	"Soy X, quiero hablar con Y" (T_1 = marca de tiempo/nonce contra repeticiones)
2	KDC → X: `E_Kmx[ Ks, E_Kmy[Ks, X], T₂ ]`	El KDC genera Ks. Se la manda a X cifrada con Kmx, junto con un "ticket" para Y cifrado con Kmy (X **no puede leerlo**)
3	X → Y: `E_Kmy[Ks, X]`	X reenvía el ticket; Y lo descifra con Kmy y obtiene Ks y la identidad de X
4	Y → X: `E_Ks[V]`	Y envía un **reto** V cifrado con Ks
5	X → Y: `E_Ks[f(V)]`	X responde con una **función del reto** → demuestra que tiene Ks (**reto-respuesta**)

Nota: Los pasos 4–5 evitan ataques de **repetición** (*replay*). Este esquema es la base de **Needham-Schroeder** y de **Kerberos**.

Mecanismos de integridad con clave simétrica

Se adjunta al mensaje un **residuo criptográfico MAC** (*Message Authentication Code*) o huella digital, que puede verificar quien tenga la **misma clave**. **HMAC** = MAC construido con una función hash.

---

11. Cifrado de clave pública

- Cada interlocutor tiene un **par de claves reversibles** generadas a la vez: `E_K1[ E_K2[L] ] = L`.
- **Seguridad computacional:** de una clave **no se puede deducir** la otra. Claves largas (1024, 2048… bits).
- Una se **protege** (**K privada**) y la otra se **publica** (**K pública**).
- Algoritmos: **RSA, DSA, ECDSA, ElGamal**…
- **Inconveniente:** longitud de clave y **tiempo de proceso**.

11.1 Confidencialidad vs autenticación (importante)

Objetivo	A cifra con…	B descifra con…	Resultado
Confidencialidad	Clave **pública de B**	Clave **privada de B**	Solo B puede leerlo
Autenticación / integridad	Clave **privada de A**	Clave **pública de A**	Cualquiera verifica que viene de A
Ambas	Privada de A y luego pública de B	Privada de B y luego pública de A	Secreto + origen garantizado

Nota: Regla: **"Para que solo lo lea B → pública de B. Para demostrar que soy A → privada de A."**

11.2 RSA (importante)

- **1977**, **Rivest, Shamir y Adleman**, sobre el modelo de Diffie-Hellman.
- Seguridad basada en la **dificultad de factorizar** grandes números.
- Cada usuario *i* tiene **Kipu** (pública) y **Kipr** (privada).

Generación de claves:

1. Elegir dos **primos grandes** p y q (> 10^{100}).
2. **n = p · q** (módulo).
3. Calcular **Φ**. La diapositiva usa **Φ = mcm(p−1, q−1)** (función de Carmichael λ); **Nota:** la versión clásica es la **función de Euler φ(n) = (p−1)(q−1)**. **Las dos funcionan.**
4. Elegir **e** sin factores comunes con p−1 y q−1 (es decir, mcd(e, Φ) = 1).
5. Calcular **d** tal que **e · d ≡ 1 (mod Φ)** (inverso modular).
6. **Clave pública = (e, n) · Clave privada = (d, n)**.

Cifrar: `C = M^e mod n` · **Descifrar:** `M = C^d mod n`

Nota: Ejemplo numérico (comprobado):

- p = 5, q = 11 → **n = 55**
- φ(n) = 4 · 10 = **40**
- e = **3** (mcd(3, 40) = 1)
- d: 3 · d ≡ 1 (mod 40) → **d = 27** (3 · 27 = 81 = 2·40 + 1 ✓)
- Pública (3, 55) · Privada (27, 55)
- Cifrar M = 2: C = 2^3 mod 55 = **8**
- Descifrar: 8^{27} mod 55 = **2** ✓
- (Con Φ = mcm(4, 10) = 20 saldría d = 7, que también funciona.)

Seguridad de RSA: para obtener d conociendo (e, n) hay que conocer Φ → hay que **factorizar n**. El mejor algoritmo citado (**Schroeppel**) necesita unos `e^√(ln n · ln ln n)` pasos:

Dígitos de n	Operaciones	Tiempo (1 µs/op)
50	$1{,}4 \cdot 10^{10}$	3,9 horas
70	$2{,}6 \cdot 10^{12}$	30 días
100	$2{,}3 \cdot 10^{15}$	74 años
200	$1{,}2 \cdot 10^{23}$	$3{,}8 \cdot 10^{9}$ años
300	$1{,}5 \cdot 10^{29}$	$4{,}9 \cdot 10^{15}$ años
500	$1{,}3 \cdot 10^{39}$	$4{,}2 \cdot 10^{25}$ años

11.3 Otros algoritmos asimétricos

Algoritmo	Clave del asunto
DSA (*Digital Signature Algorithm*)	Estándar federal EE. UU. **1994 (FIPS 186)**. **Logaritmo discreto**. **Solo firma digital**; más costoso que RSA
ECDSA / ECC (curvas elípticas)	Firma e **intercambio de claves**. **Más ligero**: clave **ECC de 256 bits ≈ RSA de 3072**. Basado en el **logaritmo discreto sobre curvas elípticas (ECDLP)**: $y^2 = x^3 + ax + b$. Curvas NIST: P-256, P-384, P-521, Curve25519…
ElGamal	**1985**, Taher ElGamal. **Logaritmo discreto**. Usado en **PGP**
LUC	**1993**, Smith y Lennon (Nueva Zelanda). **Secuencias de Lucas** Vn(p,q)

11.4 Sistemas híbridos (importante)

El asimétrico es **lento** (y más cuanto más largo es el mensaje/clave). Solución: **combinar**:

1. **Asimétrico** al inicio → **autenticar** e **intercambiar** de forma segura una clave de sesión.
2. **Simétrico** después → **confidencialidad e integridad eficientes** con esa clave.

Nota: Es exactamente lo que hacen **TLS/HTTPS, SSH, PGP y OpenVPN**.

12. Funciones hash y MAC

12.1 Funciones hash (importante)

Funciones **unidireccionales**: entrada *m* de longitud arbitraria → **resumen h = H(m) de longitud fija** (128, 160, 256… bits).

Propiedades:

- H(m) **eficiente** de calcular.
- **Resistencia a colisiones**, en tres sentidos:
  1. Dado h, difícil encontrar m con H(m) = h → **resistencia a preimagen**.
  2. Dado m_1, difícil encontrar $m_2 \neq m_1$ con $H(m_2) = H(m_1)$ → **resistencia a segunda preimagen** (colisión débil).
  3. Difícil encontrar **cualquier** par m_1, m_2 con el mismo hash → **resistencia a colisiones** (colisión fuerte).

12.2 Algoritmos

Algoritmo	Salida	Bloque	Estado
MD2, MD4	128	—	En desuso
MD5 (Rivest 1992, RFC 1321)	**128**	512	**Roto** (colisiones detectadas 1996, publicadas 2005). 4 etapas × 16 iteraciones con función T[i]
SHA-1 (1993)	**160**	512	**En desuso**
SHA-2 (SHA-224/256/384/512)	224–512	512 / 1024	**Vigente**
SHA-3 (*Keccak*)	224/256/384/512	—	**Vigente** (estructura esponja, distinta de SHA-2)
Otros	BLAKE2, RIPEMD-160, FORK-256…		

12.3 Hashes cifrados

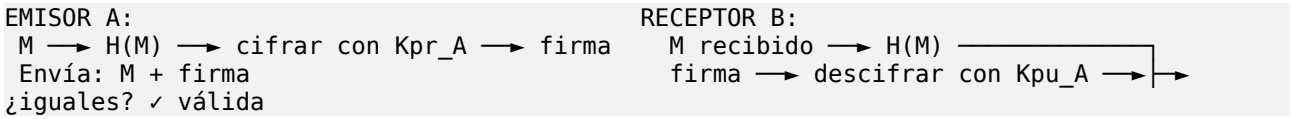
- **MAC, HMAC** (*Hash-based MAC*), **CMAC** (*Cipher-based MAC*): **códigos de integridad** con **clave simétrica**. Ej.: Poly1305-AES, SipHash.
- **AE / AEAD**: AES-GCM, OCB, SIV, **ChaCha20-Poly1305**.

Nota: Hash vs MAC vs firma: | | Clave | Integridad | Autenticación | No repudio | |---|---|---|---|---| | Hash | No | Solo contra errores | No | No | | MAC/HMAC | Simétrica compartida | ✓ | ✓ (entre los dos) | No | | Firma digital | Privada del emisor | ✓ | ✓ | ✔ |

---

13. Firma digital (importante)

Permite **autenticar de forma inequívoca al originante** y **garantizar la integridad** del mensaje. Usa el modelo **asimétrico**:



1. A calcula el **hash** del mensaje (residuo de longitud fija).
2. A **cifra el hash con su clave privada** → firma.

3. Envía mensaje + firma.
4. B **descifra la firma con la clave pública de A** y la compara con el hash que él calcula del mensaje recibido. Si coinciden → **auténtico e íntegro**.

Nota: Se firma el **hash** y no el mensaje entero porque el cifrado asimétrico es **lento**. La firma **no da confidencialidad** (el mensaje va en claro); si se quiere, se cifra además con la pública de B.

---

14. Diffie-Hellman (importante)

Permite **acordar una clave secreta a través de un canal inseguro** entre partes **sin relación previa**.

Parámetros públicos: un primo **p** y un generador **g** (conocidos por ambos). Todas las potencias son **mod p**.

14.1 DH anónimo

Alice	Visible para el atacante	Bob
Elige secreto **a**		Elige secreto **b**
$A = g^a \bmod p$	→ **A** →	
	← **B** ←	$B = g^b \bmod p$
$K = B^a = g^{ab}$		$K = A^b = g^{ab}$

El atacante ve g, p, A y B, pero obtener a o b es el **problema del logaritmo discreto** (difícil).

Nota: Ejemplo numérico (comprobado): p = 23, g = 5, a = 6, b = 15

- $A = 5^6 \bmod 23 =$ **8** · $B = 5^{15} \bmod 23 =$ **19**
- Alice: $19^6 \bmod 23 =$ **2** · Bob: $8^{15} \bmod 23 =$ **2** → **K = 2** ✓

Atención: Expuesto a ataques MitM (*Man-in-the-Middle*): un atacante en medio puede hacer un DH con Alice y otro con Bob, y ninguno lo nota, porque **no hay autenticación**.

14.2 DH autentificado

Cada uno **firma** su valor con su **clave privada** (Alice tiene priv_A y pub_B; Bob, priv_B y pub_A):

- Alice envía **A y sig_A = sign(priv_A, A)**; Bob **verifica sig_A con pub_A** (si no es válida, **aborta**).
- Bob envía **B y sig_B**; Alice verifica con pub_B. → Se evita el MitM. Ejemplos: **DHE-RSA, ECDHE-RSA** (suites TLS).

Nota: DHE = DH **efímero**: claves DH nuevas en cada sesión → **Perfect Forward Secrecy (PFS)**: si mañana roban la clave privada del servidor, **no pueden descifrar** sesiones pasadas.

---

15. Certificados

Problema: en asimétrico, cada usuario publica su clave pública. Alguien podría **suplantar** a X publicando una clave pública falsa *Kxpu* → podría descifrar lo que envíen a X y falsificar su firma.

Solución: autenticar las claves públicas. Una entidad de confianza, la **Autoridad de Certificación (CA)**, emite **certificados**: documentos electrónicos con la asociación **[Usuario, Kpu del usuario]**, **firmados digitalmente por la CA**.

15.1 Estructura del certificado

- **Identidad del usuario** (*Subject*)
- **Clave pública** del usuario
- **Fecha de emisión** y **periodo de validez**

- Otros datos: algoritmos admitidos, datos públicos, **usos permitidos** del certificado…
- **Entidad emisora** (*Issuer*, la CA)
- **Firma digital de la CA** (con su clave privada; se verifica con la pública de la CA)

15.2 Jerarquía de certificación

Problema: ¿quién acredita a la CA? Alguien podría suplantarla. **Solución: jerarquía**: una CA es avalada por otra de nivel superior, hasta una **raíz**. Dentro de una organización, la raíz es una **PCA** (*Policy Certifying Authority*).

```
CA raíz (autofirmada, de confianza)
 ├── CA intermedia 1
 │     ├── certificado de servidor
 │     └── certificado de usuario
 └── CA intermedia 2
```

→ Se desarrolla en el Anexo PKI con OpenSSL.

---

16. Chuleta final

Quiero…	Uso…
Confidencialidad rápida	**Simétrico** (AES)
Intercambiar una clave por canal inseguro	**Diffie-Hellman** (o RSA)
Integridad con clave compartida	**MAC / HMAC**
Integridad + autenticación + no repudio	**Firma digital** (hash + clave privada)
Cifrar + integridad en un paso	**AEAD** (AES-GCM, ChaCha20-Poly1305)
Fiarme de una clave pública	**Certificado** firmado por una **CA**
Lo mejor de los dos mundos	**Sistema híbrido** (asimétrico para la clave, simétrico para los datos)

Números que hay que saber: DES 64/56/16 · AES 128 / 128-192-256 / 10-12-14 · IDEA 64/128/8 · 3DES 112/168 · MD5 128 · SHA-1 160 · claves simétricas $m(m-1)/2$ vs asimétricas $2m$ · ECC 256 ≈ RSA 3072.

---

17. Preguntas y ejercicios

1. Diferencia entre ataque pasivo y activo. Pon dos ejemplos de cada.
2. ¿Qué dice el principio de Kerckhoffs?
3. ¿Cuántas claves hacen falta para que 10 usuarios se comuniquen entre sí con cifrado simétrico? ¿Y asimétrico? *Solución:* Simétrico: $10 \cdot 9/2 = 45$ · Asimétrico: $2 \cdot 10 = 20$
4. Ordena COA, CPA, KPA, CCA de menor a mayor poder del atacante.
5. ¿Por qué el cifrado de Vernam es incondicionalmente seguro? ¿Qué pasa si se reutiliza la clave?
6. Cifra "HOLA MUNDO" con Vigenère y clave "SOL". *Solución:* S=18, O=14, L=11 → H+18=Z, O+14=C, L+11=W, A+18=S, M+14=A, U+11=F, N+18=F, D+14=R, O+11=Z → **ZCWS AFFRZ**
7. ¿Por qué 2DES no duplica la seguridad de DES? ¿Por qué 3DES usa E-D-E?
8. Diferencias entre DES y AES (bloque, clave, rondas, estructura).
9. ¿Qué problema tiene ECB? Explica CBC y por qué necesita IV.
10. Describe los 5 pasos del escenario KDC. ¿Para qué sirven los pasos 4 y 5?
11. RSA: con $p = 3$, $q = 11$ y $e = 7$, calcula n, $\varphi(n)$, d, y cifra $M = 2$. *Solución:* $n = 33$ · $\varphi = 20$ · $d = 3$ ($7 \cdot 3 = 21 \equiv 1 \bmod 20$) · $C = 2^7 \bmod 33 = 128 \bmod 33 =$ **29** · comprobación: $29^3 \bmod 33 = 2$ ✓
12. Diffie-Hellman con $p = 23$, $g = 5$, $a = 4$, $b = 3$. ¿Cuál es la clave común? *Solución:* $A = 5^4 \bmod 23 = 4$ · $B = 5^3 \bmod 23 = 10$ · $K = 10^4 \bmod 23 = 4^3 \bmod 23 =$ **18**
13. ¿Por qué DH anónimo es vulnerable a MitM y cómo se soluciona?
14. Propiedades de una función hash. ¿Por qué MD5 y SHA-1 están en desuso?
15. Explica el proceso completo de firma y verificación. ¿Qué servicios proporciona? ¿Proporciona confidencialidad?

16. EtM vs E&M vs MtE. ¿Cuál es la más segura?
17. ¿Qué es un certificado, qué campos tiene y por qué hace falta una jerarquía de CAs?
18. ¿Qué es PFS y qué tipo de intercambio lo proporciona?

Bibliografía

- J.-P. Aumasson, *Serious Cryptography*, No Starch Press, 2018.
- S. Singh, *The Code Book*, Anchor, 2000.